

1. INTRODUCCIÓN

1.1. Antecedentes

En la actualidad, los pequeños negocios comerciales enfrentan desafíos significativos en la gestión de sus operaciones diarias debido a la falta de sistemas de información automatizados. La Librería 4 Hermanos, un negocio dedicado a la venta de productos diversos, ha experimentado un crecimiento sostenido en los últimos años. Sin embargo, este crecimiento ha traído consigo una serie de problemas operativos derivados del manejo manual de la información.

Actualmente, la Librería 4 Hermanos utiliza un sistema basado en registros físicos como cuadernos, libretas o archivos Excel para gestionar sus ventas, inventario y cobros. Este enfoque presenta limitaciones importantes: errores en el registro de ventas debido a que los cálculos manuales del total y el cambio son propensos a equivocaciones; inventario desactualizado porque no se sabe con certeza cuánto stock hay disponible en tiempo real, lo que genera pérdidas por faltantes o excesos de mercadería; dificultad para generar reportes, ya que la gerencia no cuenta con información oportuna para tomar decisiones como ventas diarias o productos más vendidos; y pérdida de información, puesto que los registros físicos pueden extraviarse, dañarse o perderse con el tiempo.

Por estas razones, se ha identificado la necesidad de implementar un Sistema de Ventas con Control de Inventario y Cobros en Efectivo que permita automatizar estos procesos y brindar información confiable para la toma de decisiones.

1.2. Planteamiento del Problema

La Librería 4 Hermanos actualmente maneja sus procesos de venta, inventario y cobros de manera manual, lo que genera una serie de problemas que afectan la eficiencia y rentabilidad del negocio.

Problema principal:

Ineficiencia en la gestión de ventas, inventario y cobros debido a la falta de un sistema automatizado, lo que provoca errores en los registros, pérdida de información y dificultad para tomar decisiones gerenciales oportunas.

Problemas específicos:

- En las ventas: los vendedores anotan los productos en papel, suman manualmente los totales y calculan el cambio a mano, lo que genera errores aritméticos y demoras en la atención al cliente.
- En el inventario: no existe un control preciso del stock. Los productos se descuentan de manera estimada al final del día, sin saber exactamente qué se vendió ni cuánto queda disponible. Esto provoca la venta de productos que ya no están en stock, pedidos innecesarios de productos que sí tienen suficiente cantidad y dificultad para identificar productos con bajo stock.
- En los cobros: no queda un registro confiable de los pagos recibidos, lo que dificulta la conciliación de caja al final del día y aumenta el riesgo de desfalcos o errores contables.
- En la generación de reportes: la gerencia no dispone de reportes actualizados y confiables, como ventas por día, productos más vendidos o ingresos totales, lo que impide tomar decisiones estratégicas basadas en datos reales.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar, desarrollar e implementar un sistema de ventas que permita gestionar productos, clientes, inventario y ventas, registrando los cobros realizados en efectivo y generando información útil para la toma de decisiones en la Librería 4 Hermanos.

1.3.2. Objetivos Específicos

- Analizar los requerimientos funcionales y no funcionales del sistema mediante la identificación de las necesidades de los usuarios.
- Modelar el sistema utilizando diagramas UML para visualizar la estructura y el comportamiento del sistema.
- Diseñar una base de datos relacional que almacene la información de productos, categorías, clientes, ventas, inventario y usuarios, garantizando la integridad de los datos.
- Implementar los módulos definidos utilizando un lenguaje de programación y un framework.
- Gestionar el inventario de productos mediante el control automático de entradas y salidas, asegurando que el stock se actualice en tiempo real después de cada venta.
- Registrar ventas y cobros en efectivo, calculando automáticamente el total de la venta y el cambio a devolver al cliente.
- Generar reportes básicos para apoyar la administración del negocio.

1.4. Justificación

La implementación de un sistema de ventas con control de inventario en la Librería 4 Hermanos se justifica desde varias perspectivas.

Desde el punto de vista operativo, el sistema permitirá automatizar los procesos de venta, inventario y cobros, reduciendo significativamente los errores humanos y el tiempo dedicado a tareas administrativas. Esto se traduce en una atención más rápida a los clientes y una mayor productividad del personal.

Desde el punto de vista económico, se logrará una reducción de pérdidas al tener control exacto del inventario, evitando faltantes de productos y compras innecesarias. Además, el registro automático de ventas y cobros permitirá un arqueado de caja preciso y la detección temprana de inconsistencias. Los reportes generados por el sistema brindarán información valiosa para decidir qué productos promocionar, cuándo reabastecer y cómo optimizar los recursos.

Desde el punto de vista técnico, el proyecto permitirá aplicar los conocimientos adquiridos en la asignatura de Sistemas de Información III, utilizando metodologías de desarrollo como Cascada, patrones de diseño como MVC y tecnologías actuales, fortaleciendo las competencias profesionales de los desarrolladores.

Desde el punto de vista social, el sistema contribuirá a la formalización y modernización del negocio, facilitando la generación de empleo y mejorando la experiencia de compra de los clientes al recibir comprobantes de venta claros y rápidos.

El proyecto es viable porque se cuenta con los conocimientos técnicos necesarios, las tecnologías a utilizar son de código abierto y gratuitas, y el alcance está delimitado a ventas presenciales con pagos en efectivo, lo que lo hace realizable en el tiempo establecido.

2. ANÁLISIS DEL PROBLEMA Y REQUERIMIENTOS

2.1. Usuarios del Sistema

El sistema contará con los siguientes usuarios:

2.1.1. Administrador

Es la persona encargada de la gerencia o dueño de la Librería 4 Hermanos. Es responsable de administrar la información clave del sistema.

Funciones:

- Gestionar usuarios.
- Gestionar productos.
- Gestionar categorías.
- Consultar reportes.
- Administrar inventario.

2.1.2. Cajero

Es el empleado encargado de atender al cliente y registrar las ventas.

Funciones:

- Buscar productos.
- Registrar ventas.
- Registrar cobros en efectivo.
- Emitir comprobantes de venta.
- Consultar sus propias ventas realizadas.
- Realizar arqueo de caja.

2.2. Requerimientos Funcionales

Los requerimientos funcionales definen las acciones que el sistema debe realizar para cumplir con los objetivos del negocio.

Código	Módulo	Descripción
RF-01	Inicio de Sesión	El sistema debe permitir que el usuario inicie sesión ingresando un nombre de usuario y contraseña.
RF-02	Inicio de Sesión	El sistema debe validar las credenciales contra la base de datos antes de permitir el acceso.
RF-03	Inicio de Sesión	El sistema debe mostrar un mensaje de error si las credenciales son incorrectas.
RF-04	Inicio de Sesión	El sistema debe permitir al usuario cerrar sesión en cualquier momento.

Código	Módulo	Descripción
RF-05	Gestión de Usuarios	El sistema debe permitir registrar nuevos usuarios con nombre, usuario, contraseña y rol.
RF-06	Gestión de Usuarios	El sistema debe permitir modificar los datos de un usuario existente.
RF-07	Gestión de Usuarios	El sistema debe permitir eliminar usuarios.
RF-08	Gestión de Usuarios	El sistema debe permitir consultar la lista de todos los usuarios registrados.
RF-09	Gestión de Categorías	El sistema debe permitir registrar nuevas categorías.
RF-10	Gestión de Categorías	El sistema debe permitir editar el nombre de una categoría existente.
RF-11	Gestión de Categorías	El sistema debe permitir eliminar categorías.
RF-12	Gestión de Categorías	El sistema debe permitir consultar todas las categorías registradas.
RF-13	Gestión de Productos	El sistema debe permitir crear nuevos productos con todos los campos requeridos (código, nombre, categoría, descripción, precio de venta, costo, stock disponible, stock mínimo y estado).
RF-14	Gestión de Productos	El sistema debe permitir modificar los datos de un producto existente.
RF-15	Gestión de Productos	El sistema debe permitir eliminar productos.

Código	Módulo	Descripción
RF-16	Gestión de Productos	El sistema debe permitir buscar productos por nombre, código o categoría.
RF-17	Gestión de Productos	El sistema debe permitir consultar el listado completo de productos.
RF-18	Gestión de Inventario	El sistema debe permitir consultar las existencias de cada producto.
RF-19	Gestión de Inventario	El sistema debe incrementar el stock cuando se realice una compra o reposición de productos.
RF-20	Gestión de Inventario	El sistema debe disminuir el stock automáticamente después de cada venta confirmada.
RF-21	Gestión de Inventario	El sistema debe registrar cada movimiento de inventario con fecha, producto, tipo de movimiento, cantidad y usuario responsable.
RF-22	Gestión de Inventario	El sistema debe permitir visualizar productos cuyo stock esté por debajo del stock mínimo definido.
RF-23	Gestión de Clientes	El sistema debe permitir crear nuevos clientes con nombre, documento de identidad, teléfono y dirección.
RF-24	Gestión de Clientes	El sistema debe permitir modificar los datos de un cliente existente.
RF-25	Gestión de Clientes	El sistema debe permitir eliminar clientes.
RF-26	Gestión de Clientes	El sistema debe permitir buscar clientes por nombre o documento.
RF-27	Gestión de Ventas	El sistema debe permitir crear una nueva venta seleccionando un cliente.

Código	Módulo	Descripción
RF-28	Gestión de Ventas	El sistema debe permitir buscar productos para agregarlos a la venta.
RF-29	Gestión de Ventas	El sistema debe permitir agregar productos al detalle de la venta con cantidad y precio unitario.
RF-30	Gestión de Ventas	El sistema debe permitir modificar la cantidad de un producto ya agregado.
RF-31	Gestión de Ventas	El sistema debe permitir eliminar productos del detalle de la venta.
RF-32	Gestión de Ventas	El sistema debe calcular automáticamente el subtotal de la venta.
RF-33	Gestión de Ventas	El sistema debe calcular automáticamente el total de la venta.
RF-34	Gestión de Ventas	El sistema debe permitir confirmar la venta solo si se ha registrado el cobro.
RF-35	Cobros en Efectivo	Durante la venta, el sistema debe permitir registrar el monto entregado por el cliente.
RF-36	Cobros en Efectivo	El sistema debe calcular automáticamente el cambio a devolver.
RF-37	Cobros en Efectivo	El sistema debe mostrar el cambio en pantalla antes de confirmar.
RF-38	Cobros en Efectivo	El sistema debe registrar la fecha y hora del pago al confirmar la venta.
RF-39	Cobros en Efectivo	El sistema debe validar que el monto entregado sea mayor o igual al total de la venta.

Código	Módulo	Descripción
RF-40	Comprobante de Venta	El sistema debe generar un comprobante que incluya número de venta, fecha, cliente, productos vendidos, cantidades, precios unitarios, total, monto recibido y cambio entregado.
RF-41	Comprobante de Venta	El sistema debe permitir visualizar el comprobante en pantalla después de confirmar la venta.
RF-42	Comprobante de Venta	El sistema debe permitir imprimir el comprobante.
RF-43	Reportes	El sistema debe generar un reporte de ventas del día actual.
RF-44	Reportes	El sistema debe generar un reporte de ventas por un rango de fechas.
RF-45	Reportes	El sistema debe generar un reporte de productos vendidos en un rango de fechas.
RF-46	Reportes	El sistema debe generar un reporte del inventario actualizado.
RF-47	Reportes	El sistema debe generar un reporte de productos con bajo stock.
RF-48	Reportes	El sistema debe generar un reporte de caja por un rango de fechas.

2.3. Requerimientos No Funcionales

Código	Atributo de Calidad	Descripción
RNF-01	Usabilidad	El sistema debe tener una interfaz sencilla, intuitiva y fácil de usar, con un diseño claro que no requiera entrenamiento extenso para los usuarios.
RNF-02	Rendimiento	El tiempo de respuesta para operaciones comunes debe ser inferior a 3 segundos.
RNF-03	Seguridad	El sistema debe implementar autenticación mediante usuario y contraseña para acceder a las funcionalidades. Las contraseñas deben almacenarse de forma segura.
RNF-04	Seguridad	El sistema debe validar todos los datos de entrada para evitar errores, inyecciones SQL y ataques XSS.
RNF-05	Base de Datos	El sistema debe utilizar un motor de base de datos relacional para garantizar la integridad referencial.
RNF-06	Mantenibilidad	La base de datos debe contar con un mecanismo de respaldo periódico para prevenir pérdidas de información.
RNF-07	Mantenibilidad	El código debe estar organizado, comentado y siguiendo un patrón de diseño para facilitar futuras modificaciones.
RNF-08	Arquitectura	El sistema debe seguir el modelo Cliente-Servidor, donde el navegador se comunica con el servidor web a través del protocolo HTTP.
RNF-09	Disponibilidad	El sistema debe estar disponible en el horario de atención de la Librería 4 Hermanos.
RNF-10	Usabilidad	Los mensajes de error y éxito deben ser claros y orientados al usuario, sin tecnicismos.

2.4. Casos de Uso

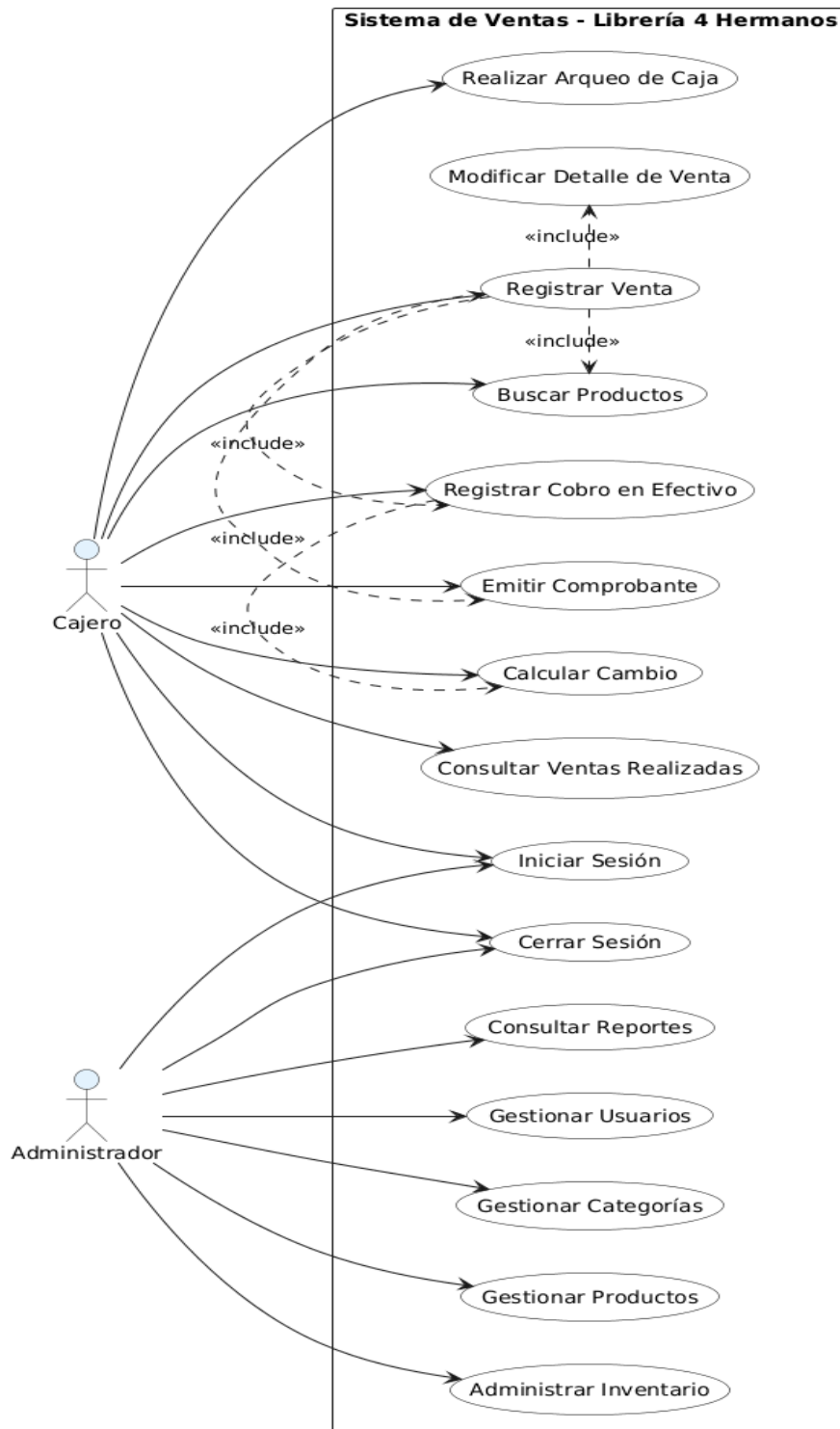
A continuación, se describen los actores y los casos de uso identificados para el sistema.

Actores:

- **Administrador:** responsable de gestionar usuarios, productos, categorías, inventario y consultar reportes.
- **Cajero:** responsable de atender al cliente, registrar ventas, gestionar cobros, emitir comprobantes, consultar sus propias ventas y realizar el arqueo de caja.

Casos de Uso por Actor:

Actor	Casos de Uso
Administrador	Iniciar sesión, Cerrar sesión, Gestionar usuarios, Gestionar categorías, Gestionar productos, Administrar inventario, Consultar reportes.
Cajero	Iniciar sesión, Cerrar sesión, Buscar productos, Registrar venta, Modificar detalle de venta, Registrar cobro en efectivo, Calcular cambio, Emitir comprobante, Consultar ventas realizadas, Realizar arqueo de caja.



6. DISEÑO DEL SISTEMA

6.1. Arquitectura del Software

Para el desarrollo del sistema de ventas de la Librería 4 Hermanos, se ha seleccionado una **Arquitectura por 3 Capas (o Tres Capas)**. Esta decisión responde a la naturaleza del sistema, que operará exclusivamente en una sola computadora dentro del establecimiento, sin conexión a Internet ni interacción con otras máquinas. El sistema es una aplicación de escritorio (standalone) que se ejecuta localmente.

¿Qué es la Arquitectura por 3 Capas?

Es un modelo de organización del código que divide la aplicación en tres niveles lógicos independientes, cada uno con una responsabilidad específica:

1. Capa de Presentación (Interfaz de Usuario)

- Es la capa que interactúa directamente con los usuarios (Administrador y Cajero). Está compuesta por las ventanas, formularios, botones, tablas y demás elementos gráficos que el usuario ve y manipula.
- Su función es **mostrar la información** y **capturar las acciones** del usuario (clics, ingreso de datos, selecciones).
- **Importante:** Esta capa **no contiene lógica de negocio ni acceso a bases de datos**. Solo se comunica con la Capa de Negocio para enviar solicitudes y recibir resultados.
- **Para nuestro sistema:** Se implementarán diferentes conjuntos de pantallas según el rol. El **Cajero** tendrá acceso a las pantallas de ventas, cobros y consulta de sus propias ventas; el **Administrador** tendrá acceso a las pantallas de gestión de usuarios, productos, categorías, inventario y reportes.

2. Capa de Negocio (Lógica del Sistema)

- Es el "cerebro" de la aplicación. Contiene todas las **reglas de negocio** y la **lógica funcional** del sistema.
- Se encarga de:
 - Validar las operaciones (por ejemplo, verificar que el stock sea suficiente antes de vender).
 - Realizar los cálculos (total de venta, cambio a devolver, actualización de inventario).
 - Aplicar las políticas del negocio (por ejemplo, no permitir ventas con monto entregado menor al total).
- Esta capa es completamente **independiente de la interfaz gráfica** y del motor de base de datos, lo que facilita las pruebas y el mantenimiento.
- **Para nuestro sistema:** Aquí residen las clases que gestionan productos, ventas, clientes, inventario y usuarios, así como los servicios que orquestan las transacciones (por

ejemplo, `ServicioVentas` que coordina la creación de una venta y la actualización del stock).

3. **Capa de Acceso a Datos (Persistencia)**

- Es la capa encargada de **interactuar con la base de datos**. Contiene las clases que ejecutan las consultas SQL (sentencias `SELECT`, `INSERT`, `UPDATE`, `DELETE`) para leer y escribir información en las tablas.
- Aísla al resto de la aplicación de los detalles específicos del motor de base de datos. Si en el futuro se decidiera cambiar de motor (por ejemplo, de SQLite a MySQL), solo se modificaría esta capa, sin afectar a las otras.
- **Para nuestro sistema:** Se utilizará un motor de base de datos local como **SQLite** o **MySQL Community Edition**. La capa de acceso a datos proveerá métodos como `ProductoDAO.obtenerPorId()`, `VentaDAO.guardar()`, etc.

Flujo de trabajo típico (ejemplo: registrar una venta):

1. El Cajero ingresa el código de un producto en la **Capa de Presentación** y hace clic en "Agregar".
2. La **Capa de Presentación** envía la solicitud a la **Capa de Negocio**.
3. La **Capa de Negocio** valida que el producto exista y que haya stock suficiente. Para ello, llama a la **Capa de Acceso a Datos** para consultar el producto y su stock.
4. La **Capa de Acceso a Datos** ejecuta una consulta SQL en la base de datos local y devuelve los datos del producto a la **Capa de Negocio**.
5. La **Capa de Negocio** procesa la información, calcula el subtotal y devuelve el resultado a la **Capa de Presentación**, que actualiza la tabla del carrito.
6. Cuando el Cajero confirma la venta, la **Capa de Presentación** envía todos los datos de la venta a la **Capa de Negocio**.
7. La **Capa de Negocio** aplica las reglas (calcula el total, verifica el monto entregado, calcula el cambio) y, si todo es correcto, ordena a la **Capa de Acceso a Datos** guardar la venta y actualizar el stock en la base de datos.
8. La **Capa de Acceso a Datos** ejecuta las sentencias SQL correspondientes y confirma la operación.
9. La **Capa de Negocio** notifica a la **Capa de Presentación** que la venta fue exitosa, y esta muestra el comprobante y el cambio al Cajero.

Ventajas de esta arquitectura para el proyecto:

- **Mantenibilidad:** Al estar las responsabilidades separadas, los cambios en una capa no afectan a las otras.

- **Reutilización:** La Capa de Negocio puede ser utilizada por diferentes interfaces.
- **Pruebas (Testing):** Se pueden probar las reglas de negocio y el acceso a datos de forma aislada, sin necesidad de la interfaz gráfica.
- **Organización del código:** El código fuente se estructura en carpetas o proyectos separados (Presentación, Negocio, Datos), lo que lo hace más legible y profesional.

6.2. Diagrama de Clases y Relaciones

El diagrama de clases representa la estructura estática del sistema, mostrando las entidades principales, sus atributos, los métodos que las operan y las relaciones entre ellas. En el contexto de la **Arquitectura por 3 Capas** definida para la Librería 4 Hermanos, se han identificado dos tipos de clases:

- **Clases de Entidad (Modelo):** Representan los objetos del mundo real (Usuario, Producto, Venta, etc.). Contienen atributos y métodos de negocio básicos y validaciones internas.
- **Clase de Servicio (Lógica de Negocio):** Representa la capa de negocio, orquestando las operaciones complejas que involucran múltiples entidades, como el registro de una venta o la generación de reportes.

A continuación, se describen las clases identificadas para el sistema:

Clase	Atributos principales	Métodos principales	Relaciones
Usuario	id, nombre, usuario, contraseña, rol	autenticar(): Valida las credenciales del usuario. cambiarContraseña(): Permite actualizar la contraseña. getRol(): Devuelve el rol del usuario (Administrador o Cajero).	Se relaciona con Venta y MovimientoInventario.

Clase	Atributos principales	Métodos principales	Relaciones
Categoría	id, nombre	<code>getNombre()</code> , <code>setNombre()</code> : Métodos de acceso básicos. <code>actualizarStock()</code> : Incrementa o decrementa la cantidad disponible.	Se relaciona con Producto (uno a muchos).
Producto	id, código, nombre, descripción, precio_venta, costo, stock, stock_minimo, estado	<code>verificarStockDisponible()</code> : Verifica si hay suficiente stock para una venta. <code>calcularPrecioConMargen()</code> : Calcula el precio final aplicando márgenes (si aplica). <code>estaActivo()</code> : Retorna si el producto está disponible para la venta.	Se relaciona con Categoría (muchos a uno) y con DetalleVenta y MovimientoInventario (uno a muchos).
Cliente	id, nombre, documento, teléfono, dirección	<code>getNombreCompleto()</code> : Devuelve el nombre completo del cliente. <code>validarDocumento()</code> : Valida el formato del documento de identidad.	Se relaciona con Venta (uno a muchos).
Venta	id, fecha, total, estado	<code>calcularTotal()</code> : Suma los subtotales de todos los detalles de la venta.	Se relaciona con Cliente (muchos a uno), Usuario (muchos a uno) y

Clase	Atributos principales	Métodos principales	Relaciones
		agregarDetalle(): Añade un producto al carrito de compras. confirmarVenta(): Cambia el estado de la venta a "Confirmada". generarComprobante(): Genera el texto o formato del ticket de compra.	DetalleVenta (uno a muchos).
DetalleVenta	id, cantidad, precio_unitario, subtotal	calcularSubtotal(): Multiplica cantidad por precio unitario. getTotalDetalle(): Devuelve el subtotal calculado.	Se relaciona con Venta (muchos a uno) y Producto (muchos a uno).
MovimientoInventario	id, fecha, tipo (Entrada/Salida), cantidad	getTipoMovimiento(): Retorna si es entrada o salida. getCantidad(): Retorna la cantidad movida.	Se relaciona con Producto (muchos a uno) y Usuario (muchos a uno).
ServicioVentas (Clase de Negocio)	(No tiene atributos propios, solo dependencias)	registrarVenta(): Orquesta todo el proceso: valida stock, crea la venta, guarda detalles, actualiza inventario y registra el cobro. buscarProducto():	Depende (usa) de las clases Venta, Producto, Usuario y DetalleVenta para ejecutar la lógica de negocio.

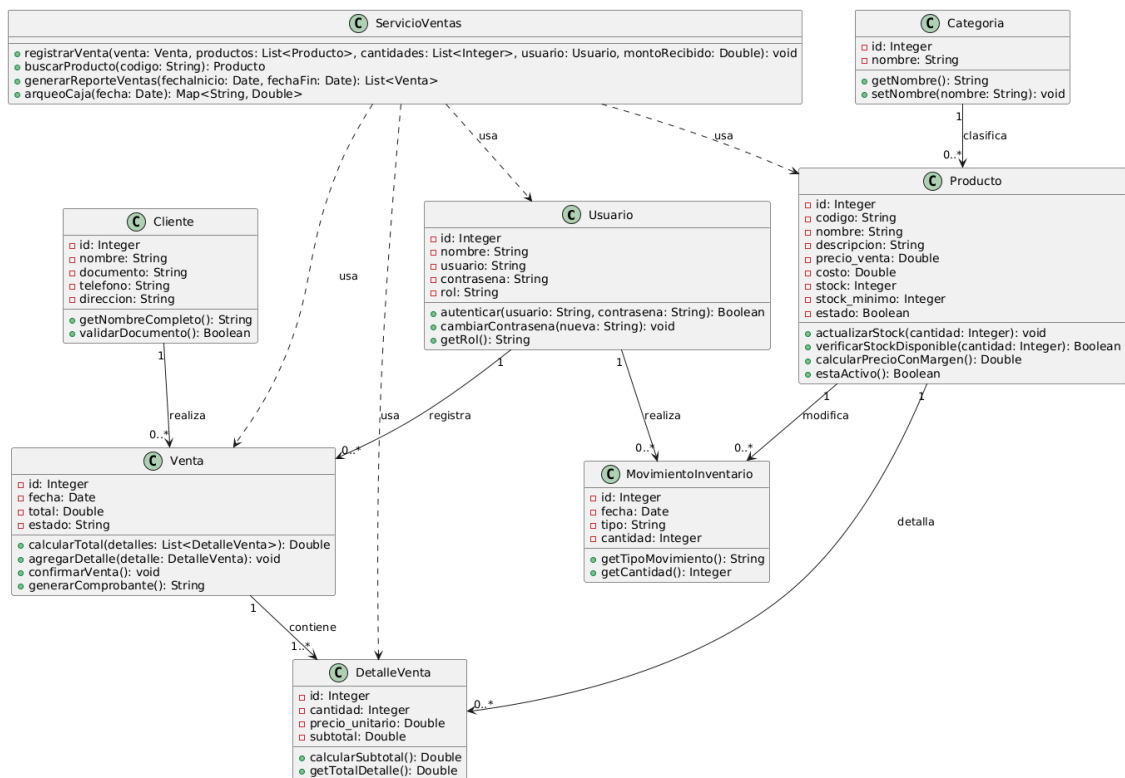
Clase	Atributos principales	Métodos principales	Relaciones
		Consulta productos por código o nombre. <code>generarReporteVentas()</code>: Genera reportes en un rango de fechas. <code>arquearCaja()</code>: Calcula el total de ventas, recibido y cambios para el cierre de caja.	

Relaciones principales:

- Un **Usuario** (Administrador o Cajero) registra varias **Ventas** y varios **Movimientos de Inventario**.
- Una **Categoría** agrupa varios **Productos**.
- Un **Producto** pertenece a una única **Categoría**.
- Un **Cliente** realiza varias **Ventas**.
- Una **Venta** contiene uno o varios **Detalles de Venta**.
- Un **Detalle de Venta** corresponde a un único **Producto**.
- Un **Producto** tiene varios **Movimientos de Inventario**.
- La clase **ServicioVentas** actúa como el orquestador de la Capa de Negocio, utilizando las entidades para realizar las operaciones solicitadas por la Capa de Presentación.

Justificación de la inclusión de la clase ServicioVentas:

En una arquitectura por 3 capas, la **Capa de Negocio** debe estar aislada de la interfaz de usuario y del acceso a datos. La clase `ServicioVentas` centraliza la lógica compleja (como registrar una venta, que implica múltiples inserciones en tablas, actualización de stock y control de transacciones), evitando que esta lógica se disperse en las clases de entidad o en la interfaz gráfica. Esto mejora la mantenibilidad y facilita las pruebas unitarias.



6.3. Modelo de Base de Datos (MER y DER)

El modelo de datos constituye el cimiento sobre el cual se almacenará y gestionará toda la información del sistema. Para el desarrollo de la Librería 4 Hermanos, se ha optado por un **Modelo Entidad-Relación (MER)** que define las entidades y sus relaciones, el cual se traduce posteriormente en un **Diagrama Entidad-Relación (DER)** que representa la estructura lógica de la base de datos relacional.

Modelo Entidad-Relación (MER):

El MER conceptual identifica las siguientes entidades principales y sus relaciones:

- **Usuarios:** Representa a las personas que operan el sistema (Administrador y Cajero).
- **Categorías:** Agrupa los productos por tipo o familia.
- **Productos:** Representa los artículos que se venden en la librería.
- **Clientes:** Registra la información de las personas que compran en el negocio.
- **Ventas:** Almacena el encabezado de cada transacción de compra.
- **DetalleVentas:** Contiene los ítems específicos que componen cada venta.
- **MovimientosInventario:** Registra el historial de entradas y salidas de cada producto.

Diagrama Entidad-Relación (DER):

El DER traduce el modelo conceptual a un esquema lógico de tablas relacionales. A continuación, se detalla la estructura de cada tabla, sus campos, tipos de datos, claves primarias y claves foráneas:

Tabla	Campos	Clave Primaria	Claves Foráneas
Usuarios	id (INTEGER), nombre (VARCHAR), usuario (VARCHAR), contrasena (VARCHAR), rol (VARCHAR)	id	-
Categorias	id (INTEGER), nombre (VARCHAR)	id	-
Productos	id (INTEGER), codigo (VARCHAR), nombre (VARCHAR), descripcion (TEXT), precio_venta (DECIMAL), costo (DECIMAL), stock (INTEGER), stock_minimo (INTEGER), estado (BOOLEAN), categoria_id (INTEGER)	id	categoria_id → Categorias(id)
Clientes	id (INTEGER), nombre (VARCHAR), documento (VARCHAR), telefono (VARCHAR),	id	-

Tabla	Campos	Clave Primaria	Claves Foráneas
	direccion (VARCHAR)		
Ventas	id (INTEGER), fecha (DATETIME), total (DECIMAL), estado (VARCHAR), cliente_id (INTEGER), usuario_id (INTEGER)	id	cliente_id → Clientes(id), usuario_id → Usuarios(id)
DetalleVentas	id (INTEGER), cantidad (INTEGER), precio_unitario (DECIMAL), subtotal (DECIMAL), venta_id (INTEGER), producto_id (INTEGER)	id	venta_id → Ventas(id), producto_id → Productos(id)
MovimientosInventario	id (INTEGER), fecha (DATETIME), tipo (VARCHAR), cantidad (INTEGER), producto_id (INTEGER), usuario_id (INTEGER)	id	producto_id → Productos(id), usuario_id → Usuarios(id)

Relaciones e integridad referencial:

- Un **Usuario** puede registrar muchas **Ventas** y muchos **Movimientos de Inventario**.
- Una **Categoría** puede agrupar muchos **Productos**.
- Un **Producto** pertenece a una única **Categoría**.
- Un **Cliente** puede realizar muchas **Ventas**.
- Una **Venta** se compone de uno o varios **Detalles de Venta**.
- Un **Detalle de Venta** corresponde a un único **Producto**.

- Un **Producto** puede tener muchos **Movimientos de Inventario** que registran su flujo de stock.

